Project 1 Report: Information Exposure Maximization

Student ID: 12411454

1. Problem Statement

The task is to solve Information Exposure Maximization (IEM) under the Independent Cascade model. We are given a directed graph `G = (V, E)`, two campaigns with edge probabilities `p1` and `p2`, two initial seed sets `I1` and `I2`, and an additional budget `k`. We need to choose two extra seed sets `S1` and `S2` such that `|S1| + |S2| <= k`.

Let `R1` and `R2` be the reached sets of the two campaigns after diffusion. The objective is to maximize the expected number of balanced vertices, that is, vertices reached by both campaigns or by neither campaign. Equivalently, we minimize the expected size of the symmetric difference between `R1` and `R2`.

In this project I used only course-level methods: Monte Carlo evaluation, greedy construction, local search, and simulated annealing. The main goal of the final version is stable scoring rather than aggressive overfitting to released cases.

2. Evaluator

2.1 Main idea

`Evaluator.py` estimates the objective value of a given balanced seed set by Monte Carlo simulation on the full graph. In each sampled world:

1. campaign 1 diffuses from `I1 union S1`;
2. campaign 2 diffuses from `I2 union S2`;
3. the evaluator counts balanced vertices.

This is a direct implementation of the problem definition.

2.2 Implementation details

The diffusion process is simulated by a queue-based IC procedure. In the implementation, a vertex is marked as reached for a campaign once it is exposed by an activated in-neighbor, while only successful activation trials continue the queue. The objective is then computed from the mismatch between the two reached sets.

To reduce runtime, I used:

- bitset-style simulation for small graphs;
- `bytearray` workspaces for large graphs;
- touched-node reset instead of clearing the full array each time;
- online mean and variance;
- adaptive early stopping based on the 95% confidence interval.

2.3 Evaluator pseudocode

```
Evaluator(G, I1, I2, S1, S2):
    U1 <- I1 union S1
    U2 <- I2 union S2
    plan <- choose sample plan by graph size
    stats <- empty online statistics

    while sample count < plan.max:
        R1 <- diffuse campaign 1 from U1
        R2 <- diffuse campaign 2 from U2
        value <- |V| - mismatch(R1, R2)
        stats.update(value)
        if sample count >= plan.min and confidence interval is small enough:
            break

    return stats.mean
```

3. Heuristic Solver

3.1 Design

`IEMP_Heur.py` starts from a greedy framework, but it does not evaluate every node at every step. The final design has four parts:

1. preprocessing;
2. candidate-pool construction;
3. greedy construction under several budget splits;
4. local improvement and timeout protection.

3.2 Preprocessing and candidate pools

For each node, the solver computes several cheap statistics, including weighted out-strength, one-hop coverage, approximate two-hop coverage, and common-node strength. These values are only heuristics, but they are useful for pruning.

At each step, the solver builds a candidate pool from several ranked lists:

- strong nodes for campaign 1;
- strong nodes for campaign 2;
- nodes that look good for both campaigns;
- nodes near the initial seeds;
- repair candidates for reducing imbalance.

This avoids repeated full-graph scanning.

3.3 Budget split and greedy construction

The budget is not always best used in the same way, so the solver tries several split profiles of the form:

- campaign-1 only;
- campaign-2 only;
- common-style additions to both sides.

Every profile satisfies `a + b + 2c <= k`.

For one profile, the solver repeatedly:

1. builds a candidate pool;
2. ranks actions by a cheap proxy score;
3. keeps only a small shortlist;
4. reranks the shortlist with shared random worlds;
5. applies the best action.

The shared-world comparison is important because it reduces Monte Carlo noise when comparing similar actions.

3.4 Local search and guard

After construction, the solver runs a small local search with neighborhoods such as swap, move, mirror, and pair replace. If there is not enough time for full local search, it runs a very short last-mile refinement instead.

To avoid losing points on hidden tests, the solver also includes:

- a fast timeout baseline;
- a stronger guard baseline;
- graph-dependent time limits;
- legal fallback output on timeout or unexpected exceptions.

3.5 Heuristic pseudocode

```
HeuristicSolver(G, I1, I2, k):
    timer <- create graph-dependent timer
    baseline <- build guard baseline
    preprocess node strengths and rankings
    profiles <- generate promising budget splits

    best <- baseline

    for each profile while time remains:
        state <- empty additional seeds
        while profile not full and time remains:
            pool <- build candidate pool
            shortlist <- top actions by cheap score
            action <- rerank shortlist by shared random worlds
            apply action
        update best if current profile is better
```

```
17
18      if enough time:
19          best <- local search(best)
20      else if a small safe slack remains:
21          best <- last-mile refinement(best)
22
23      return best legal solution
```

4. Evolutionary / Simulated Annealing Solver

4.1 Design

`IEMP_Evol.py` is implemented as a simulated annealing solver. A solution is stored as two sparse seed sets, one for each campaign. This makes add, delete, swap, and common-node moves easy to implement.

The solver uses the heuristic method as a warm start. This is important because a purely random start is unstable on this problem.

4.2 Main components

The SA solver contains the following components:

1. a timeout baseline and a guard baseline;
2. heuristic warm start;
3. structured neighborhoods:
    - add,
    - delete,
    - swap,
    - move between campaigns,
    - commonize,
    - de-commonize;
4. two-level evaluation:
    - fast evaluation for screening,
    - accurate evaluation for elite candidates;
5. a short last-mile annealing burst near the end.

To save time, both fast and accurate evaluations are cached by solution signature.

4.3 Annealing process

For each start solution, the solver generates a small batch of neighbors, keeps the most promising ones by fast evaluation, then uses accurate evaluation on only a few candidates. Improvements are always accepted; worsening moves are accepted with the standard Metropolis rule.

I kept the final version conservative: one start for small graphs and one start for large graphs. In earlier experiments, more restarts made runtime much larger but did not improve the average score.

4.4 SA pseudocode

```
SASolver(G, I1, I2, k):
    timer <- create graph-dependent timer
    baseline <- build timeout baseline
    warm <- heuristic warm start
    starts <- generate start solutions from baseline, warm, and
perturbations
    best <- best evaluated start

    for each selected start while time remains:
        current <- start
        temperature <- initialize by graph size

        for a bounded number of iterations:
            if time is tight:
                break
            neighbors <- generate structured neighbors
            shortlist <- keep best neighbors by fast evaluation
            candidate <- best of shortlist by accurate evaluation
            if accept(candidate, current, temperature):
                current <- candidate
            if current better than best:
                best <- current
            temperature <- temperature * cooling

    if a small safe slack remains:
        best <- short last-mile annealing(best)

    return best legal solution
```

5. Key Implementation Choices

The final score depends not only on solution quality but also on stable runtime and legal output. The following choices were the most useful in practice:

- shared random worlds for fairer candidate comparison;
- multi-stage screening instead of full expensive evaluation everywhere;
- reusable workspaces in the evaluator;
- small local search instead of deep neighborhood exploration;
- explicit timer guard and fallback output.

These are all implementation-level improvements built on top of lecture methods.

6. Parameter Settings

Only the final important parameters are listed here.

6.1 Evaluator

- small graph samples: `256..768`, batch `64`;
- medium graph samples: `96..224`, batch `32`;
- dense graph samples: `80..160`, batch `16`.

6.2 Heuristic

- candidate pool limit: `180` for small graphs, `240` for large graphs;
- shared-world stages: `(8, 12, 20)` for small graphs, `(8, 10, 12)` for large graphs;
- beam width / depth: `3 / 3` for small graphs, `2 / 2` for large graphs;
- local search iterations: `3` for small graphs, `1` for large graphs;
- last-mile minimum slack: `0.25s` for small graphs, `3.0s` for large graphs.

6.3 Simulated Annealing

- max iterations: `90` for small graphs, `70` for large graphs;
- fast evaluation worlds: `12` for small graphs, `4` for large graphs;
- accurate evaluation worlds: `32` for small graphs, `8` for large graphs;
- candidate limit: `96` for small graphs, `128` for large graphs;
- start count: `1` for both small and large graphs;
- last-mile minimum slack: `0.45s` for small graphs, `2.4s` for large graphs.

These values were selected by repeated local experiments, with average stability as the first priority.

7. Complexity

For the evaluator, if `R` is the number of Monte Carlo samples, the worst-case time complexity is `O(R * (|V| + |E|))`, and the space complexity is `O(|V| + |E|)`.

For the heuristic solver, preprocessing is close to linear in graph size in practice, and the main search cost is controlled by candidate-pool pruning, small shortlists, and bounded local search.

For the SA solver, the dominant cost comes from repeated neighbor evaluation. If `T` is the number of iterations, `B` is the neighbor batch size, and `W` is the number of sampled worlds used in evaluation, the runtime is roughly proportional to `T * B * W * diffusion_cost`, but caching and strict iteration limits reduce the actual cost.

8. Experimental Results

8.1 Released cases

Final local results on the released test cases are shown below.

Heuristic

Case	Objective	Time
map1	456.3671875	25.046 s
map2	35923.7395833	69.033 s
map3	36025.8984375	66.040 s

Evolutionary / SA

Case	Objective	Time
map1	455.46875	28.774 s
map2	13808.28125	31.586 s
map3	13778.1354167	30.196 s

All released heuristic and evolutionary cases exceeded the published higher targets locally, so the released solver subtotal is `7.8 / 7.8`.

8.2 Stress tests

I also tested several larger synthetic cases to check robustness.

Heuristic stress tests

Instance	Size	Objective	Time
huge sparse	`50000 / 90000`	49941.8020833	162.089 s
extreme dense (guarded)	`9000 / 180000`	8999.8625	208.356 s

Without the timer guard, the extreme dense case still had not finished after about `604s` on the local machine.

Evolutionary stress tests

Instance	Size	Objective	Time
large dense	`13984 / 85000`	13025.8883929	148.258 s
extreme dense	`3454 / 70000`	3453.4479167	154.435 s

These tests suggest that the SA solver is usually faster on dense graphs, while the heuristic solver is slightly stronger on some instances.

9. Limitations and Conclusion

The final code is strong and stable locally, but it still has some limitations. First, all three programs rely on Monte Carlo evaluation, so there is still some random variation. Second, the heuristic uses approximate ranking and pruning, so it may miss some globally good actions. Third, the hidden thresholds are unknown, so local released performance does not guarantee hidden bonuses.

Overall, the final system meets the project requirements and stays within the scope of the course. The evaluator is correct and efficient, the heuristic solver is strong and time-aware, and the simulated annealing solver improves upon a good warm start while keeping runtime under control.

References

1. K. Garimella, A. Gionis, N. Parotsidis, N. Tatti. *Balancing information exposure in social networks*. NeurIPS 2017.
2. S. Cheng, H. Shen, J. Huang, W. Chen, X. Cheng. *IMRank: influence maximization via finding self-consistent ranking*. SIGIR 2014.